PQC CBOM Scanner

Cryptographic Bill of Materials

Quantum Readiness Assessment Tool

Software Requirements Specification

IEEE Std 830-1998 Compliant

Document Version	1.0.0
Date	2026-03-20
Status	Final Draft
Project	quantumCheck
Hackathon	PNB Cybersecurity Hackathon 2025–26
Theme	Quantum-Proof Systems

Contents

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms, and Abbreviations . . . 1
1.4 References . . . 2
1.5 Document Overview . . . 2

2 Overall Description **3**
2.1 Product Perspective . . . 3
2.1.1 Context Diagram . . . 3
2.1.2 Relationship to Existing Systems . . . 3
2.2 Product Functions . . . 4
2.3 User Classes and Characteristics . . . 4
2.4 Operating Environment . . . 5
2.5 Design and Implementation Constraints . . . 5
2.6 Assumptions and Dependencies . . . 5

3 Functional Requirements **6**
3.1 Feature 1: Host Input and Scan Initiation . . . 6
3.2 Feature 2: TLS Scanning Engine . . . 7
3.3 Feature 3: Quantum Readiness Classification . . . 7
3.3.1 Classification Tiers . . . 8
3.4 Feature 4: Subdomain Discovery . . . 9
3.5 Feature 5: CBOM JSON Output . . . 9
3.6 Feature 6: Interactive HTML Report . . . 10
3.7 Feature 7: Web GUI and Real-Time Streaming . . . 11
3.8 Feature 8: Report Lifecycle Management . . . 12

4 External Interface Requirements **13**
4.1 User Interfaces . . . 13
4.1.1 Web GUI . . . 13
4.1.2 HTML Report Interface . . . 14
4.1.3 Command-Line Interface . . . 14
4.2 Hardware Interfaces . . . 15
4.3 Software Interfaces . . . 15
4.4 Communication Interfaces . . . 15

5 Non-Functional Requirements **17**
5.1 Performance Requirements . . . 17
5.2 Security Requirements . . . 17
5.3 Reliability and Availability . . . 18
5.4 Maintainability . . . 18
5.5 Portability . . . 18
5.6 Usability . . . 19

6 System Architecture **20**
6.1 Architectural Overview . . . 20

6.2 Component Diagram . . . 20
6.3 Data Flow . . . 21
6.3.1 Scan Pipeline . . . 21
6.3.2 CBOM Assembly . . . 21
6.4 Deployment Architecture . . . 22
6.4.1 Docker Container . . . 22
6.4.2 Concurrency Model . . . 22
6.5 Key Algorithmic Decisions . . . 23
6.5.1 Root Domain Approximation . . . 23
6.5.2 Severity Ordering for Domain Groups . . . 23
6.5.3 JSON Escaping in Bash . . . 23

7 Other Requirements and Constraints **24**
7.1 Legal and Ethical Constraints . . . 24
7.2 Standards Compliance . . . 24
7.3 Resource Constraints . . . 24
7.4 Known Limitations . . . 25
7.5 Future Enhancements (Roadmap) . . . 25

8 Appendix **26**
8.1 CBOM JSON Schema . . . 26
8.2 Quantum Classification Decision Table . . . 27
8.3 Environment Variables Reference . . . 27
8.4 Document Revision History . . . 28
8.5 Glossary of Quantum Computing Threats . . . 28

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **PQC CBOM Scanner** — a web-based cybersecurity assessment tool that audits public-facing servers for post-quantum cryptography (PQC) readiness.

The document is intended for the following audiences:

- **Development Team** — as a binding technical specification during implementation and testing.
- **Security Evaluators** — as a baseline for verifying that the system correctly classifies TLS configurations against NIST PQC standards.
- **Hackathon Judges / Reviewers** — as a structured description of the project scope, design decisions, and requirements.
- **End Users** — as a reference for understanding system capabilities and expected behaviour.

1.2 Scope

The **PQC CBOM Scanner** (project codename: *quantumCheck*) is a full-stack cybersecurity tool that:

1. Establishes TLS handshakes with one or more user-supplied hostnames.
2. Extracts a complete **Cryptographic Bill of Materials (CBOM)** from each TLS session, including cipher suite, key exchange mechanism, certificate details, and signature algorithms.
3. Classifies each scanned asset against a four-tier quantum readiness framework aligned to NIST FIPS 203, 204, and 205.
4. Generates a machine-readable JSON CBOM and a self-contained interactive HTML dashboard report.
5. Optionally discovers and scans subdomains of a root domain using the Subfinder tool.

The system is delivered as a **Dockerised web application** (Flask / Gunicorn) with a dual interface: a browser-based GUI and a command-line interface.

Out of scope:

- Active exploitation or penetration testing of target hosts.
- Certificate Authority (CA) management or certificate issuance.
- Remediation automation (the tool provides guidance, not automated fixes).
- Support for non-TLS protocols (SSH, IPsec, etc.) in v1.0.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
CBOM	*Cryptographic Bill of Materials* — a structured inventory of all cryptographic primitives used in a system or communication session.
PQC	*Post-Quantum Cryptography* — cryptographic algorithms designed to resist attacks from quantum computers.
TLS	*Transport Layer Security* — the standard protocol for encrypted communication over a network.

Term	Definition
NIST	*National Institute of Standards and Technology* — the US federal body responsible for cryptographic standards.
FIPS	*Federal Information Processing Standard* — e.g. FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA).
ML-KEM	*Module-Lattice-based Key Encapsulation Mechanism* (Kyber) — a NIST-standardised PQC key exchange algorithm.
ML-DSA	*Module-Lattice-based Digital Signature Algorithm* (Dilithium) — a NIST-standardised PQC signature scheme.
SLH-DSA	*Stateless Hash-Based Digital Signature Algorithm* (SPHINCS+) — a NIST-standardised PQC signature scheme.
FN-DSA	*Falcon Digital Signature Algorithm* — another NIST PQC signature standard.
ECDSA	*Elliptic Curve Digital Signature Algorithm.*
ECDH	*Elliptic Curve Diffie-Hellman* key exchange.
RSA	*Rivest–Shamir–Adleman* public-key cryptosystem.
HNDL	*Harvest Now, Decrypt Later* — a threat model where adversaries collect encrypted traffic today to decrypt it once quantum computers mature.
FQS	*Fully Quantum Safe* — highest readiness tier.
SSE	*Server-Sent Events* — a web API for streaming data from server to client.
GUI	*Graphical User Interface.*
CLI	*Command-Line Interface.*
WSGI	*Web Server Gateway Interface* — Python web server standard.
SRS	*Software Requirements Specification.*

1.4 References

[1] NIST FIPS 203 — *Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, August 2024.
[2] NIST FIPS 204 — *Module-Lattice-Based Digital Signature Standard (ML-DSA)*, August 2024.
[3] NIST FIPS 205 — *Stateless Hash-Based Digital Signature Standard (SLH-DSA)*, August 2024.
[4] IEEE Std 830-1998 — *IEEE Recommended Practice for Software Requirements Specifications.*
[5] RFC 8446 — *The Transport Layer Security (TLS) Protocol Version 1.3*, IETF, 2018.
[6] OWASP Cryptographic Failures — *OWASP Top 10 2021, A02.*
[7] ProjectDiscovery Subfinder — `https://github.com/projectdiscovery/subfinder`.
[8] NIST NCCoE Migration to Post-Quantum Cryptography Project, SP 1800-38.

1.5 Document Overview

The remainder of this SRS is structured as follows:

Chapter 2 provides a product overview, user classes, operating environment, and assumptions.
Chapter 3 specifies all functional requirements grouped by system feature.
Chapter 4 describes user, hardware, software, and communication interfaces.
Chapter 5 defines non-functional requirements (performance, security, reliability, portability).
Chapter 6 presents the system architecture, data flow, and deployment model.
Chapter 7 lists design constraints, legal considerations, and dependencies.
Chapter 8 contains the CBOM JSON schema, quantum classification logic table, and version history.

2 Overall Description

2.1 Product Perspective

The PQC CBOM Scanner is a **standalone, self-contained web application** that operates as an active TLS interrogation tool. It does not depend on any external monitoring infrastructure, SIEM, or certificate inventory service. The system sits entirely within a single Docker container and communicates outward only to the user-specified target hosts.

2.1.1 Context Diagram

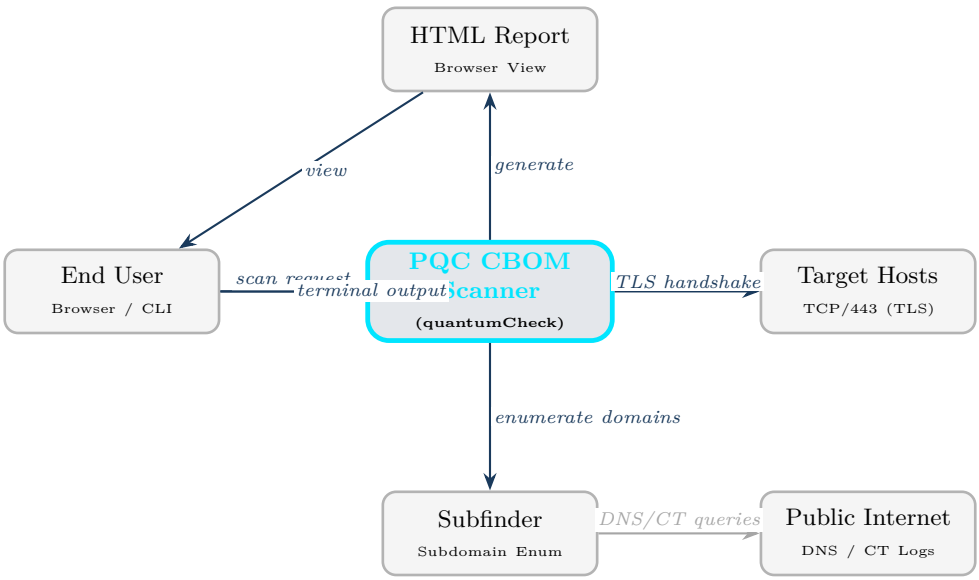


Figure 2.1: System Context Diagram — PQC CBOM Scanner and its external entities

The system interacts with three external entities:

1. **End User** — accesses the web GUI via a browser or invokes the CLI scanner directly.
2. **Target Hosts** — public-facing web servers whose TLS configuration is assessed. The scanner initiates outbound connections on TCP port 443 (or user-specified ports).
3. **Subfinder Service** — an optional embedded binary that performs passive and active subdomain enumeration by querying public DNS and certificate transparency logs.

2.1.2 Relationship to Existing Systems

The tool is positioned as a *pre-migration assessment utility* in a broader PQC migration workflow:

2.2 Product Functions

At a high level, the PQC CBOM Scanner provides the following functions:

Table 2.1: High-Level Product Functions

#	Function	Description
F1	TLS Scanning	Establish TLS handshakes and extract cipher, key exchange, and certificate metadata.
F2	Subdomain Discovery	Enumerate subdomains of a root domain and include them in the scan scope.
F3	Quantum Assessment	Classify each asset against a four-tier PQC readiness framework.
F4	CBOM Generation	Produce a structured JSON Cryptographic Bill of Materials.
F5	HTML Report	Render a self-contained interactive HTML dashboard summarising findings.
F6	Web GUI	Provide a browser-based interface for scan configuration, execution, and result viewing.
F7	Scan Control	Allow users to start and stop scans in real time.

2.3 User Classes and Characteristics

Table 2.2: User Classes

User Class	Technical Level	Primary Use Case
Security Analyst	High	Perform bulk TLS audits, interpret CBOM JSON output, integrate into workflows.
System Administrator	Medium	Identify vulnerable servers before compliance deadlines; use Web GUI.
CISO / Security Manager	Low	Review HTML dashboard to gauge organisation-wide quantum readiness.
Developer / DevOps	Medium	Run CLI scanner as part of CI/CD pipelines or infrastructure testing.
Hackathon Evaluator	Varies	Assess tool correctness, UX, and alignment to NIST standards.

2.4 Operating Environment

Table 2.3: Operating Environment

Component	Requirement
Host OS	Any OS capable of running Docker (Linux, macOS, Windows with WSL2).
Container Runtime	Docker Engine $\geq$ 20.10 or compatible OCI runtime.
Python Version	Python 3.11 (within container).
Shell Environment	Bash 5.x (within container).
OpenSSL Version	OpenSSL $\geq$ 3.0 (within container).
Network Access	Outbound TCP/443 (or custom port) to target hosts from the container.
Browser (Web GUI)	Any modern browser (Chrome $\geq$ 90, Firefox $\geq$ 88, Edge $\geq$ 90).
Display Resolution	$\geq$ 1280×720 recommended for optimal dashboard layout.

2.5 Design and Implementation Constraints

- The scanner relies on `openssl s_client` and therefore inherits its support matrix for TLS extensions and PQC algorithms. OpenSSL versions below 3.5 may not report ML-KEM key exchange natively.
- The web server uses a **single Gunicorn worker** to ensure that in-memory scan and report state is consistent across concurrent requests.
- Long-running scans may exceed browser SSE connection timeouts; a 2-hour hard deadline is enforced server-side.
- Subdomain discovery requires the embedded Subfinder binary (`bin/subfinder`); if absent, subdomain discovery is silently skipped.
- The system does not persist data to disk beyond the `output/` directory; there is no database backend.

2.6 Assumptions and Dependencies

- Target hosts are reachable via the network from the Docker container.
- Target hosts use TCP port 443 by default (overridable via `host:port` syntax).
- The OpenSSL binary included in the container supports TLS 1.3.
- The Subfinder binary bundled in the image is version 2.13.0 (Linux x86_64); alternative architectures require rebuilding the image.
- The user has basic familiarity with web browsers and network hostnames.
- The tool assumes good-faith use; it does not implement rate limiting or access control beyond OS-level process permissions.

3 Functional Requirements

Requirements are identified with the prefix **FR-** (Functional Requirement). Priority levels: **M** = Must Have, **S** = Should Have, **C** = Could Have.

3.1 Feature 1: Host Input and Scan Initiation

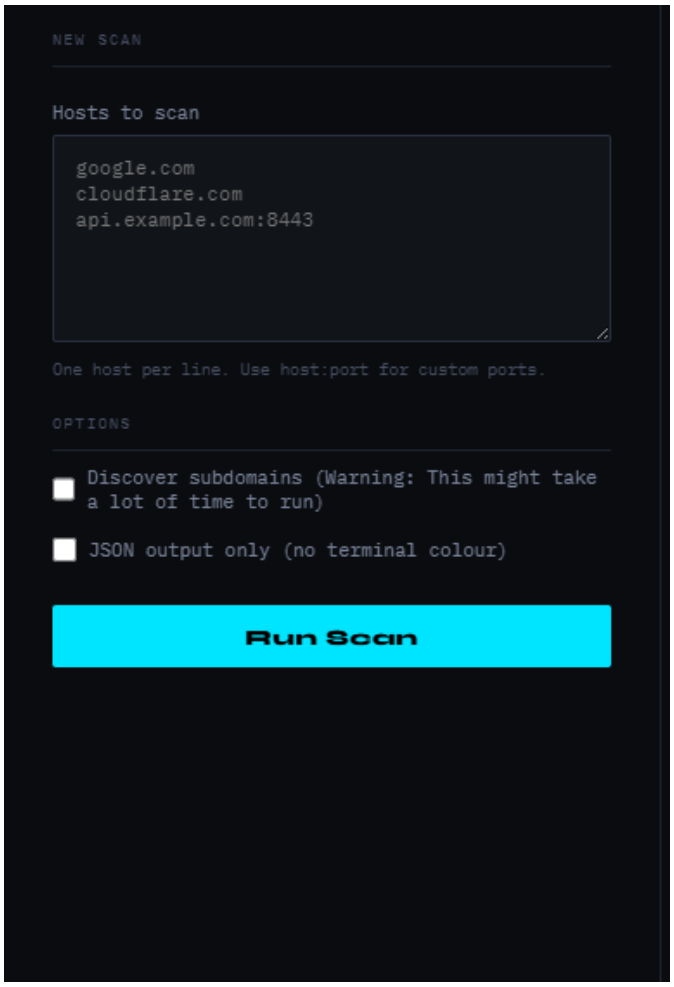


Figure 3.1: Web GUI — Input panel with host list and scan options

FR-1.1 — Multi-Host Input

The system **shall** accept one or more hostnames or IP addresses from the user, one per line, supporting the optional `host:port` syntax to override the default port of 443. **[M]**

FR-1.2 — File-Based Input (CLI)

The CLI scanner **shall** accept a plain-text file of hosts via the `-f <file>` flag, skipping blank lines

and lines beginning with `#`. **[M]**

FR-1.3 — Scan Configuration Options

The Web GUI **shall** expose at minimum the following boolean options prior to scan start:

- **Discover Subdomains** — invoke Subfinder before scanning.
- **JSON Output Only** — suppress ANSI colour codes in terminal output.

[M]

FR-1.4 — Scan Button State Management

The *Run Scan* button **shall** be disabled while a scan is in progress and re-enabled upon completion or cancellation to prevent duplicate scans. **[M]**

3.2 Feature 2: TLS Scanning Engine

FR-2.1 — TLS Handshake Initiation

For each host in the scan list, the system **shall** initiate a TLS 1.3 handshake using `openssl s_client` with a per-host timeout of 8 seconds. **[M]**

FR-2.2 — TLS Version Extraction

The system **shall** extract and record the negotiated TLS protocol version (TLS 1.0, 1.1, 1.2, or 1.3). **[M]**

FR-2.3 — Cipher Suite Extraction

The system **shall** extract the negotiated cipher suite name (e.g. `TLS_AES_256_GCM_SHA384`). **[M]**

FR-2.4 — Key Exchange Extraction

The system **shall** extract the key exchange algorithm from the TLS handshake, including hybrid PQC variants (e.g. `X25519Kyber768`, `ML-KEM-768`). **[M]**

FR-2.5 — Certificate Chain Extraction

The system **shall** extract the following fields from the server's leaf certificate:

- Subject Distinguished Name
- Issuer Distinguished Name
- Public key algorithm and bit-length
- Signature algorithm
- Validity period (not-before, not-after)
- Certificate chain depth

[M]

FR-2.6 — Connection Failure Handling

If the TLS handshake fails (timeout, refused, unreachable), the system **shall** record a `connection_error` entry in the CBOM for that host rather than aborting the entire scan. **[M]**

FR-2.7 — Certificate Verification Status

The system **shall** record whether the server certificate chain verification succeeded or failed during the handshake. **[S]**

3.3 Feature 3: Quantum Readiness Classification

3.3.1 Classification Tiers

Table 3.1: Quantum Readiness Tiers

#	Label	Symbol	Criteria
1	Fully Quantum Safe	★ **FQS**	PQC key exchange *and* ML-DSA/SLH-DSA/FN-DSA certificate *and* AES-256 or ChaCha20 symmetric cipher.
2	PQC Ready	▼ **PQC**	PQC key exchange *and* safe symmetric cipher, but classical (RSA/ECDSA) certificate.
3	Not PQC Ready	◇ **NOT**	Classical key exchange (X25519, ECDH, RSA-KEM) *or* AES-128 symmetric cipher (marginal security).
4	Critical	✗ **CRIT**	Broken symmetric cipher: 3DES, RC4, DES, or export-grade.

FR-3.1 — Key Exchange Classification

The system **shall** classify the key exchange as:

- `pqc_hybrid` — X25519Kyber768 or similar hybrid.
- `pqc_pure` — pure ML-KEM or equivalent.
- `classical` — X25519, ECDH, secp256r1, RSA, etc.
- `unknown` — could not be determined from handshake output.

[M]

FR-3.2 — Certificate Classification

The system **shall** classify the server certificate algorithm as:

- `pqc` — ML-DSA, SLH-DSA, FN-DSA (Falcon).
- `ecdsa` — EC-based signature.
- `rsa` — RSA-based signature.
- `unknown` — unrecognised algorithm.

[M]

FR-3.3 — Symmetric Cipher Classification

The system **shall** classify the negotiated symmetric cipher as:

- `safe` — AES-256-GCM, ChaCha20-Poly1305.
- `marginal` — AES-128-GCM (weakened to 64-bit by Grover's algorithm against a quantum adversary).
- `broken` — 3DES, RC4, DES, NULL, EXPORT ciphers.
- `unknown` — unrecognised cipher suite.

[M]

FR-3.4 — Composite Quantum Label Assignment

The system **shall** assign one of the four tiers defined in Table 3.1 to each scanned host based on the combination of key exchange, certificate, and symmetric cipher classifications. **[M]**

FR-3.5 — Vulnerability List Generation

For each host, the system **shall** generate a human-readable list of specific vulnerabilities identified, including:

- Key exchange susceptibility to Shor's algorithm.
- RSA/ECDSA certificate susceptibility to Shor's algorithm.
- AES-128 bit-security reduction by Grover's algorithm.
- Broken cipher suites (3DES, RC4, etc.).
- Certificate expiry (expired or < 30 days remaining).

[M]

FR-3.6 — Actionable Recommendations

The system **shall** generate context-aware, label-specific remediation recommendations for each host, referencing concrete migration steps (e.g. enabling X25519Kyber768, upgrading to ML-DSA certificate, disabling 3DES). **[M]**

3.4 Feature 4: Subdomain Discovery

FR-4.1 — Subfinder Integration

When the *Discover Subdomains* option is enabled and the Subfinder binary is present, the system **shall** execute Subfinder against the root domain of each supplied host with a 120-second timeout. **[S]**

FR-4.2 — Subdomain Merging

Discovered subdomains **shall** be merged into the scan host list and scanned with the same TLS pipeline as explicit hosts. **[S]**

FR-4.3 — Graceful Degradation

If Subfinder is not available, the system **shall** silently skip subdomain discovery and proceed with only the user-supplied hosts. **[M]**

3.5 Feature 5: CBOM JSON Output

FR-5.1 — Machine-Readable CBOM

The system **shall** produce a JSON CBOM document conforming to the schema defined in Appendix 8.1. **[M]**

FR-5.2 — CBOM Metadata

The CBOM **shall** include top-level metadata: scan start timestamp, scan end timestamp, and total host count. **[M]**

FR-5.3 — Per-Asset CBOM Entry

Each scanned host **shall** produce a CBOM asset entry containing at minimum: hostname, port, TLS version, cipher suite, key exchange (type and algorithm), certificate details, symmetric cipher, quantum label, vulnerability list, recommendations, and scan timestamp. **[M]**

FR-5.4 — JSON File Persistence

The system **shall** write the final CBOM JSON to the `output/` directory with a timestamp-based filename. **[S]**

3.6 Feature 6: Interactive HTML Report

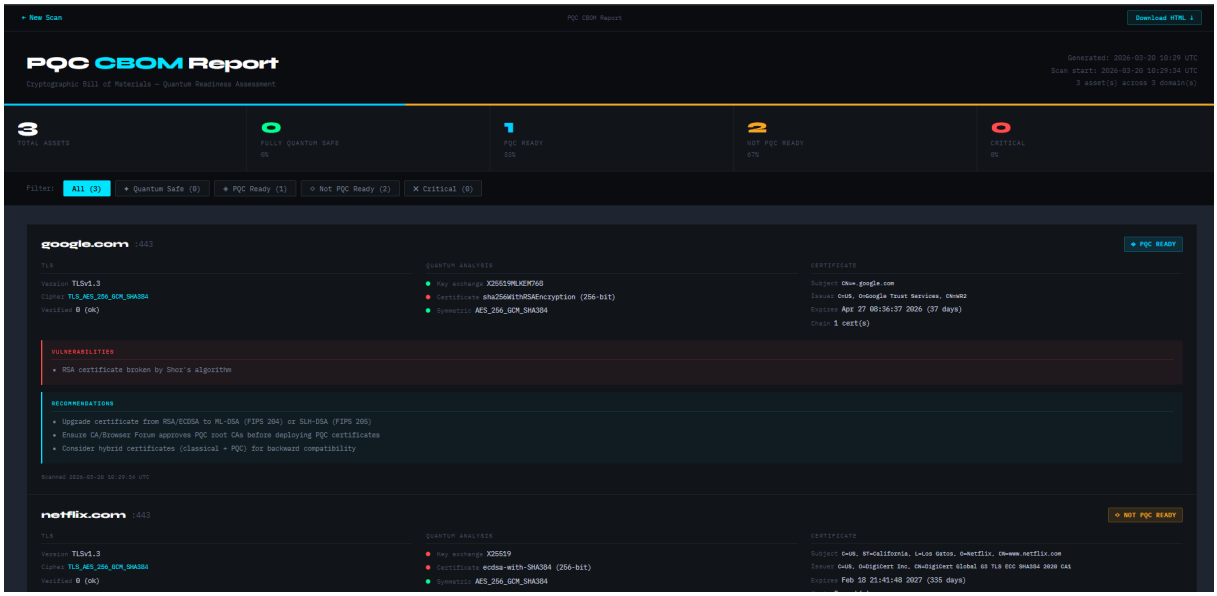


Figure 3.2: HTML Report — Stats bar, progress bar, and asset cards

FR-6.1 — Self-Contained HTML

The generated report **shall** be a single self-contained HTML file with all CSS and JavaScript embedded inline, requiring no internet connection to render. **[M]**

FR-6.2 — Summary Statistics

The report header **shall** display total asset count and per-tier count with percentage for each of the four quantum readiness labels. **[M]**

FR-6.3 — Visual Progress Bar

The report **shall** render a horizontal progress bar with colour-coded segments proportional to the count of each tier, providing an at-a-glance portfolio health view. **[M]**

FR-6.4 — Filter Controls

The report **shall** provide interactive filter buttons to show only assets matching a selected tier label (All / FQS / PQC / Not PQC / Critical) without a page reload. **[M]**

FR-6.5 — Domain Grouping

Assets **shall** be grouped by their root domain (eTLD+1), with subdomains displayed in a collapsible drawer beneath the root asset card. **[S]**

FR-6.6 — Asset Card Detail

Each asset card **shall** display:

- TLS version and cipher suite.
- Key exchange algorithm and PQC classification dot.
- Certificate subject, issuer, expiry (colour-coded), and chain depth.
- Signature algorithm and PQC classification dot.
- Symmetric cipher and safety dot.
- Vulnerability list.

- Recommendation list.
- Scan timestamp.

[M]

FR-6.7 — Certificate Expiry Warnings

Certificate expiry **shall** be displayed in:

- Red — certificate already expired.
- Amber — fewer than 30 days remaining.
- Normal colour — otherwise.

[M]

FR-6.8 — Report Download

The report page **shall** provide a *Download HTML* button that saves the report as a standalone file to the user's local machine. [S]

3.7 Feature 7: Web GUI and Real-Time Streaming

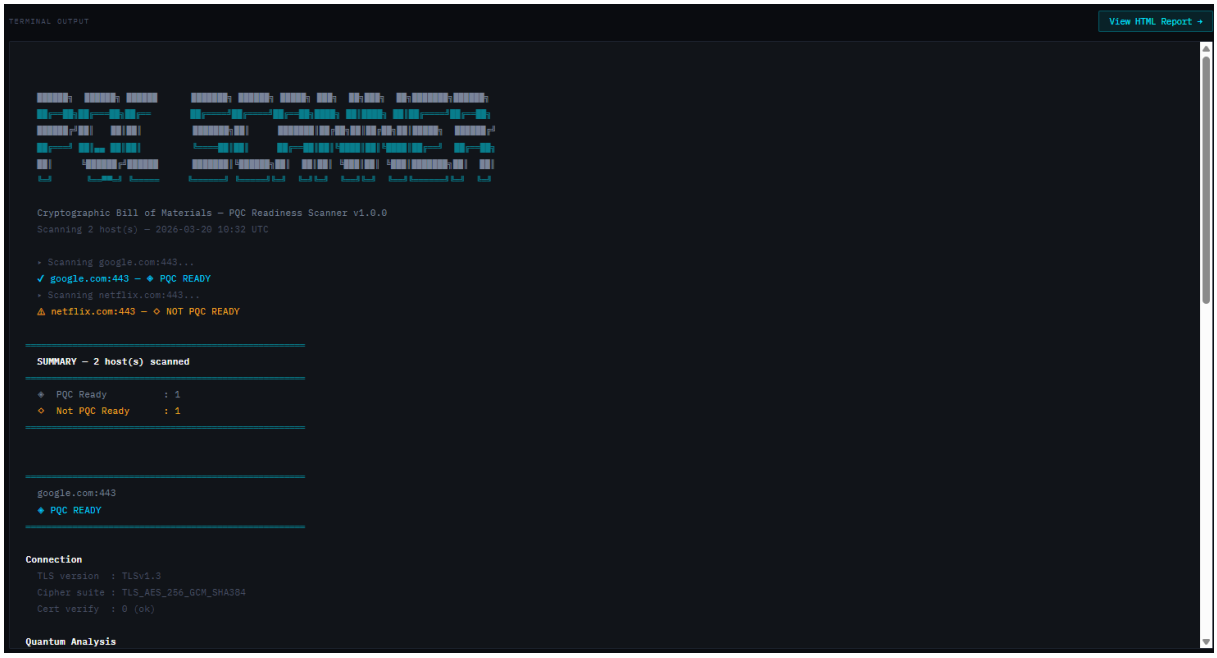


Figure 3.3: Web GUI — Terminal output panel with real-time SSE streaming

FR-7.1 — Real-Time Output Streaming

The web application **shall** stream scan output to the browser in real time using Server-Sent Events (SSE), rendering each line as it is produced by the scanner. [M]

FR-7.2 — ANSI Colour Rendering

Colour codes emitted by the scanner (
`033[...m` sequences) **shall** be converted to appropriate HTML/CSS styling so that tier labels appear in their corresponding colours in the browser output panel. [M]

FR-7.3 — Scan Status Indicator

A spinner or progress indicator **shall** be visible while a scan is running and hidden upon completion. **[S]**

FR-7.4 — Report Link Post-Scan

Upon successful scan completion, the GUI **shall** display a *View HTML Report* link that opens the generated report in a new browser tab. **[M]**

FR-7.5 — Stop Scan

The GUI **shall** provide a *Stop Scan* button that becomes visible during a running scan and, when clicked, sends a SIGTERM to the scanner process to terminate it gracefully. **[M]**

FR-7.6 — SSE Keepalive

The SSE stream **shall** emit a keepalive comment every 5 seconds to prevent intermediate proxies from closing the connection prematurely. **[S]**

3.8 Feature 8: Report Lifecycle Management

FR-8.1 — In-Memory Report Storage

Generated reports **shall** be held in server memory and accessible via a unique report identifier for the duration of the configured TTL (default: 3600 seconds). **[M]**

FR-8.2 — Automatic Report Eviction

A background daemon thread **shall** evict expired reports every 5 minutes to prevent unbounded memory growth. **[M]**

FR-8.3 — TTL Configuration

The report TTL **shall** be configurable via the `REPORT_TTL` environment variable without code changes. **[S]**

4 External Interface Requirements

4.1 User Interfaces

4.1.1 Web GUI

The primary user interface is a single-page web application served at `http://localhost:5000` (or the configured `PORT`).

Layout Overview

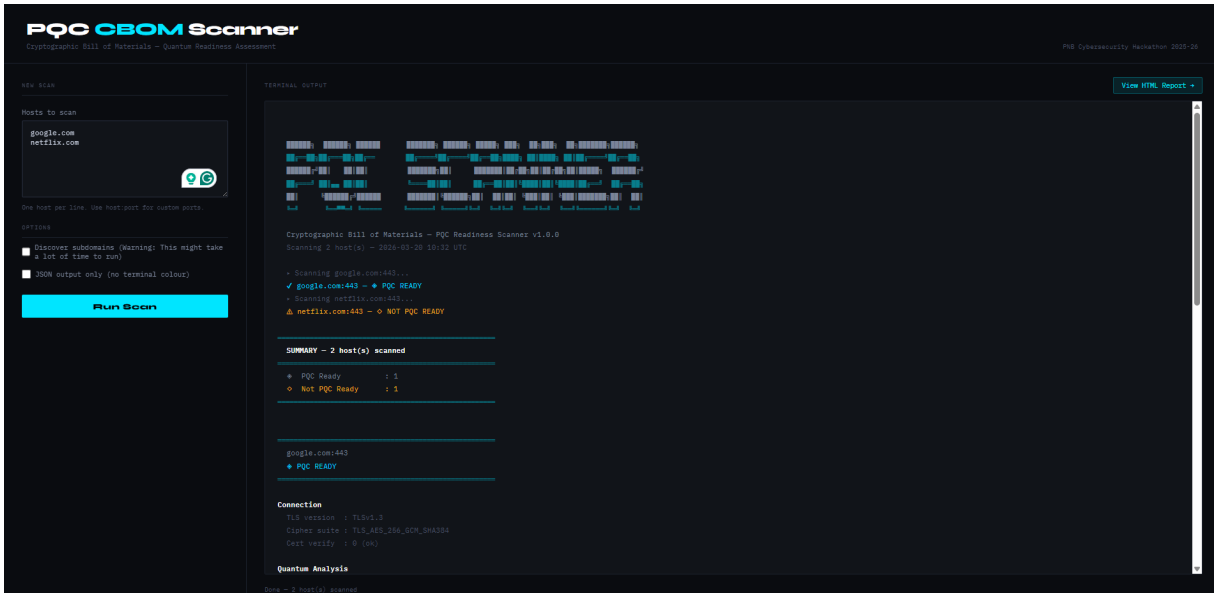


Figure 4.1: Web GUI — Full layout: header, input panel (left), and output panel (right)

The page is divided into three regions:

Header Application title (*PQC CBOM Scanner*) with a cyan accent on *CBOM*, subtitle, and hackathon context badge.

Left Panel (Input) Host textarea, option checkboxes, Run Scan and Stop Scan buttons, version info footer.

Right Panel (Output) Terminal-style scrollable output area with real-time SSE content, scan status spinner, and post-scan report link.

Colour Scheme

Table 4.1: UI Colour Conventions

Element	Hex Value	Usage
Background	`#0A0C10`	Page and panel backgrounds.
Cyan Accent	`#00E5FF`	Headings, buttons, borders.
Fully Quantum Safe	`#00FF94`	FQS label, dots, text.
PQC Ready	`#00BFFF`	PQC label, dots, text.
Not PQC Ready	`#F5A623`	NOT label, warnings, dots.
Critical	`#FF4D4D`	CRIT label, errors, stop button.

Typography

- **Syne** (Google Fonts) — used for headings and labels.
- **IBM Plex Mono** (Google Fonts) — used for monospace terminal output, code, and cipher names.

4.1.2 HTML Report Interface

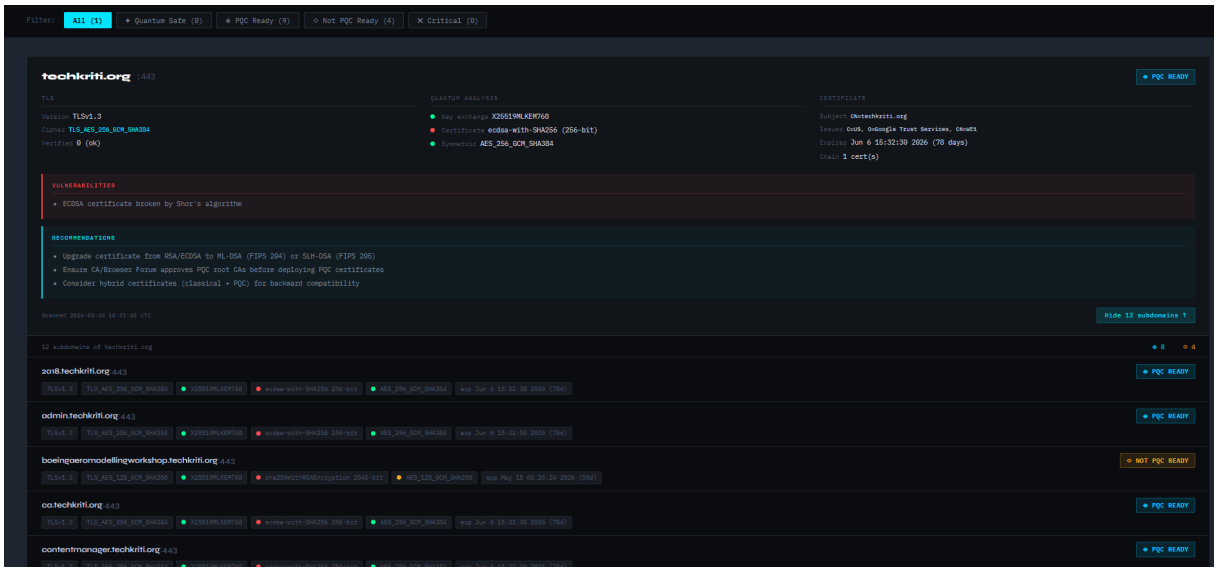


Figure 4.2: HTML Report — Filter bar and domain group with subdomain drawer expanded

The generated report provides the following interactive UI elements:

- **Sticky navigation bar** — Back to New Scan link and Download HTML button always visible while scrolling.
- **Statistics cells** — large numerals showing count and percentage per tier.
- **Progress bar** — colour-segmented horizontal bar.
- **Filter buttons** — toggle visibility of asset cards by tier.
- **Domain groups** — root asset card + collapsible subdomain drawer with compact subdomain rows.
- **Asset cards** — full CBOM detail per host with colour-coded indicator dots.

4.1.3 Command-Line Interface

The CLI scanner (`scan.sh`) accepts the following arguments:

Table 4.2: CLI Arguments

Argument	Description
`<host>`	One or more space-separated hostnames.
`-f <file>`	Read hosts from a plain-text file.
`-subdomains`	Enable Subfinder subdomain discovery.
`-json`	Output clean JSON; suppress ANSI codes.
`-report`	Generate an HTML report after scanning.

4.2 Hardware Interfaces

The application has no direct hardware interface requirements. It runs entirely in software and communicates over standard Ethernet/Wi-Fi network interfaces via the host OS's TCP/IP stack. No special hardware (HSM, SmartCard, etc.) is required.

4.3 Software Interfaces

Table 4.3: Software Interface Dependencies

Component	Version	Role
Python	3.11	Application runtime.
Flask	$\geq$ 3.0	HTTP framework; routes, SSE, template rendering.
Gunicorn	$\geq$ 21.0	Production WSGI server (gthread worker).
OpenSSL	$\geq$ 3.0	TLS handshake and certificate parsing via `s_client`.
Bash	5.x	Scanner script (`scan.sh`); host loop, JSON assembly.
Subfinder	2.13.0	Passive/active subdomain enumeration.
Docker	$\geq$ 20.10	Container packaging and runtime.

4.4 Communication Interfaces

Table 4.4: Communication Interfaces

Interface	Protocol	Description
Browser $\leftrightarrow$ App	HTTP/1.1	Form submission, page requests, report download.
Browser $\leftarrow$ App (SSE)	HTTP/1.1	Server-Sent Events stream of scan output lines.
App $\rightarrow$ Target Hosts	TLS 1.3 (TCP/443)	Outbound TLS handshake for interrogation.
App $\rightarrow$ Subfinder	STDIN/STDOUT	Local IPC pipe; domain input and subdomain output.
Subfinder $\rightarrow$ Internet	DNS / HTTPS	Queries public DNS resolvers and certificate transparency logs.

Port assignments:

- `5000/TCP` — Web GUI HTTP server (configurable via `PORT` env var).

- `443/TCP` — Default outbound port for TLS target connections (overridable per host).

5 Non-Functional Requirements

Non-functional requirements are identified with the prefix **NFR-**.

5.1 Performance Requirements

NFR-P.1 — Per-Host Scan Timeout

Each TLS handshake attempt **shall** complete or time out within **8 seconds**, ensuring predictable throughput when scanning large host lists. **[M]**

NFR-P.2 — Subdomain Discovery Timeout

Subfinder execution per root domain **shall** be capped at **120 seconds** to prevent the overall scan from hanging indefinitely. **[M]**

NFR-P.3 — SSE Stream Hard Deadline

Each Server-Sent Events stream **shall** be terminated after a maximum of **2 hours** server-side to prevent resource leaks from abandoned browser connections. **[M]**

NFR-P.4 — Concurrent Request Capacity

The Gunicorn configuration with **1 worker and 16 threads shall** support at least 8 simultaneous SSE connections and associated background scan jobs without degradation. **[S]**

NFR-P.5 — Report Generation Latency

HTML report generation from a complete CBOM JSON **shall** complete within **5 seconds** for scan results containing up to 500 hosts. **[S]**

NFR-P.6 — UI Responsiveness

The web GUI **shall** remain responsive (non-blocking user interaction) during active scanning; SSE streaming **shall not** freeze the browser tab. **[M]**

5.2 Security Requirements

NFR-S.1 — No Persistent Secret Storage

The system **shall not** store, log, or transmit any private keys, session tokens, or user credentials. The application is read-only with respect to target host data. **[M]**

NFR-S.2 — Input Sanitisation

Host input provided by the user **shall** be validated and sanitised before being passed to shell commands to prevent command injection. **[M]**

NFR-S.3 — No Destructive Operations on Targets

The scanner **shall** perform only passive TLS interrogation (initiating a standard TLS handshake). No vulnerability exploitation, fuzzing, or denial of service techniques **shall** be employed. **[M]**

NFR-S.4 — Container Isolation

When deployed via Docker, the application **shall** run as a non-root user inside the container to minimise the blast radius of any security vulnerability. **[S]**

NFR-S.5 — CBOM Data Confidentiality

CBOM JSON files written to `output/` **shall** be readable only by the OS user running the container. No report **shall** be exposed to the public internet without explicit user action. **[S]**

NFR-S.6 — XSS Prevention in Report

All host-derived data embedded in the HTML report **shall** be HTML-escaped to prevent cross-site scripting (XSS) if the report is viewed in a shared browser context. **[M]**

5.3 Reliability and Availability

NFR-R.1 — Graceful Connection Failure Handling

A failed TLS connection to any individual host **shall not** abort the scan of remaining hosts; the failure **shall** be recorded in the CBOM and scanning **shall** continue. **[M]**

NFR-R.2 — JSON Output Validity

The CBOM JSON produced by the scanner **shall** always be valid, well-formed JSON. Special characters in hostnames, certificate fields, or cipher names **shall** be properly escaped. **[M]**

NFR-R.3 — Report Generation on Partial Failure

If one or more hosts in a scan produce errors, the HTML report **shall** still be generated for all hosts that were successfully scanned. **[M]**

NFR-R.4 — Memory Leak Prevention

The background report reaper thread **shall** prevent unbounded accumulation of in-memory reports by evicting entries that exceed the TTL. **[M]**

5.4 Maintainability

NFR-M.1 — Modular Architecture

The system **shall** maintain a clear separation of concerns:

- `scan.sh` — TLS scanning and CBOM JSON generation.
- `app.py` — Web server, SSE streaming, process management.
- `generate_report.py` — HTML report rendering.

[M]

NFR-M.2 — Classification Logic Isolation

Quantum readiness classification logic **shall** be implemented in a single, clearly labelled block within `scan.sh` so that updates to NIST standards can be applied by editing one location. **[S]**

NFR-M.3 — Environment-Based Configuration

All tunable parameters (port, TTL, output path) **shall** be configurable via environment variables with sensible defaults, requiring no code changes for deployment customisation. **[S]**

5.5 Portability

NFR-PO.1 — Docker Portability

The application **shall** be fully self-contained within its Docker image, including all runtime dependencies (Python, Flask, OpenSSL, Bash, Subfinder), requiring only Docker Engine on the host. **[M]**

NFR-PO.2 — Architecture Note

The bundled Subfinder binary targets Linux x86_64. For ARM64 or other architectures, the Dockerfile **shall** be rebuilt with the appropriate binary. The rest of the application **shall** be architecture-agnostic. **[S]**

NFR-PO.3 — Browser Compatibility

The web GUI and HTML report **shall** render correctly in all evergreen browsers (Chrome, Firefox, Edge, Safari) without requiring browser plugins or extensions. **[M]**

5.6 Usability

NFR-U.1 — Colour-Coded Tier Display

All four quantum readiness tiers **shall** be visually distinguished by consistent colour coding across the terminal output, HTML report, and asset cards (Table 4.1). **[M]**

NFR-U.2 — Zero-Installation Web Access

A user **shall** be able to use the full Web GUI without installing any software beyond Docker; no Node.js, npm, or frontend build step is required. **[M]**

NFR-U.3 — Offline Report Viewing

A downloaded HTML report **shall** be fully viewable offline without any network request. **[M]**

6 System Architecture

6.1 Architectural Overview

The PQC CBOM Scanner follows a **three-tier architecture** within a single Docker container:

1. **Presentation Tier** — Flask-served HTML/JS/CSS web GUI and generated HTML report.
2. **Application Tier** — Flask application (`app.py`) managing scan orchestration, SSE streaming, process lifecycle, and report storage.
3. **Processing Tier** — Bash scanner (`scan.sh`) performing TLS interrogation and CBOM construction; Python report generator (`generate_report.py`).

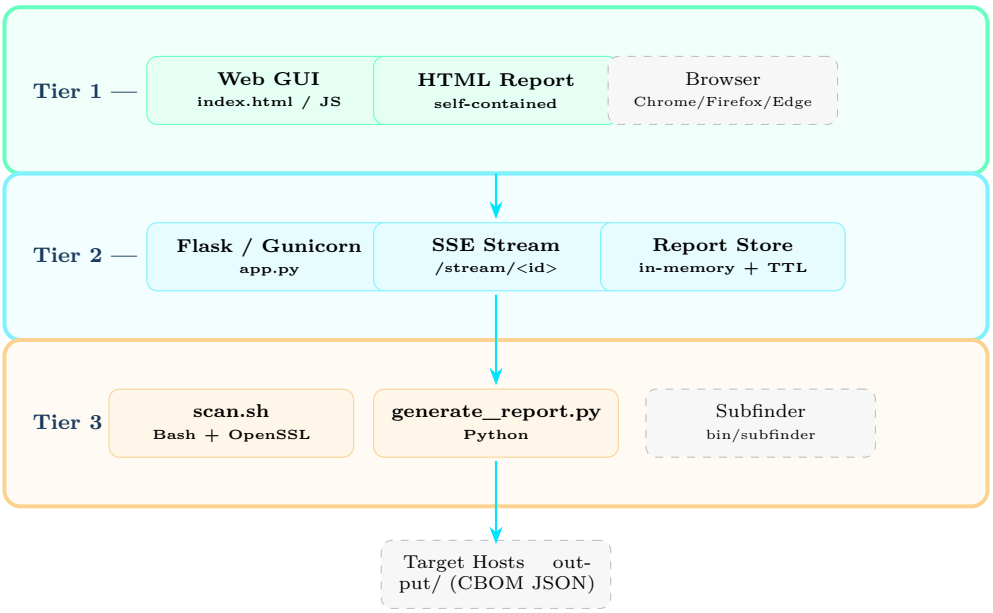
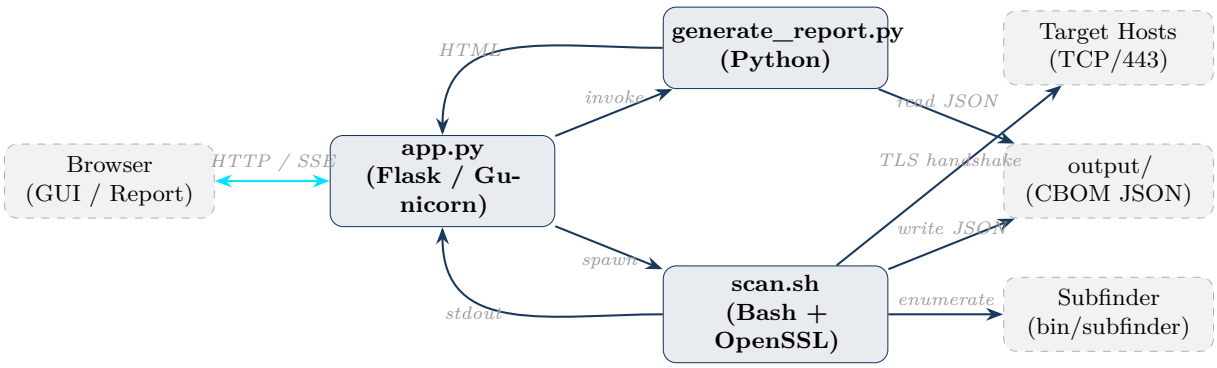


Figure 6.1: System Architecture Diagram — Three-tier component overview

6.2 Component Diagram



6.3 Data Flow

6.3.1 Scan Pipeline

The following sequence describes the end-to-end data flow for a typical web GUI scan:

Step 1. User submits host list and options via the Web GUI form (`POST /scan`).
Step 2. `app.py` assigns a unique `scan_id`, stores options in memory, and spawns `scan.sh` as a subprocess.
Step 3. Browser opens an SSE connection to `GET /stream/<scan_id>`.
Step 4. `scan.sh` (optionally) runs Subfinder to expand the host list.
Step 5. For each host, `scan.sh` invokes `openssl s_client` and parses the TLS handshake output.
Step 6. Parsed fields are classified (key exchange, certificate, symmetric) and a quantum label is assigned.
Step 7. Each host's CBOM entry (JSON) is appended to the CBOM file in `output/`.
Step 8. Human-readable output lines are written to `stdout`; Flask reads these and forwards each line as an SSE event to the browser.
Step 9. After all hosts are processed, `scan.sh` emits a `done` marker.
Step 10. Flask reads the CBOM JSON, passes it to `generate_report.py`, and stores the rendered HTML in memory under a `report_id`.
Step 11. Flask emits a `report` SSE event containing the `report_id`.
Step 12. Browser displays a *View Report* link pointing to `GET /view/<report_id>`.

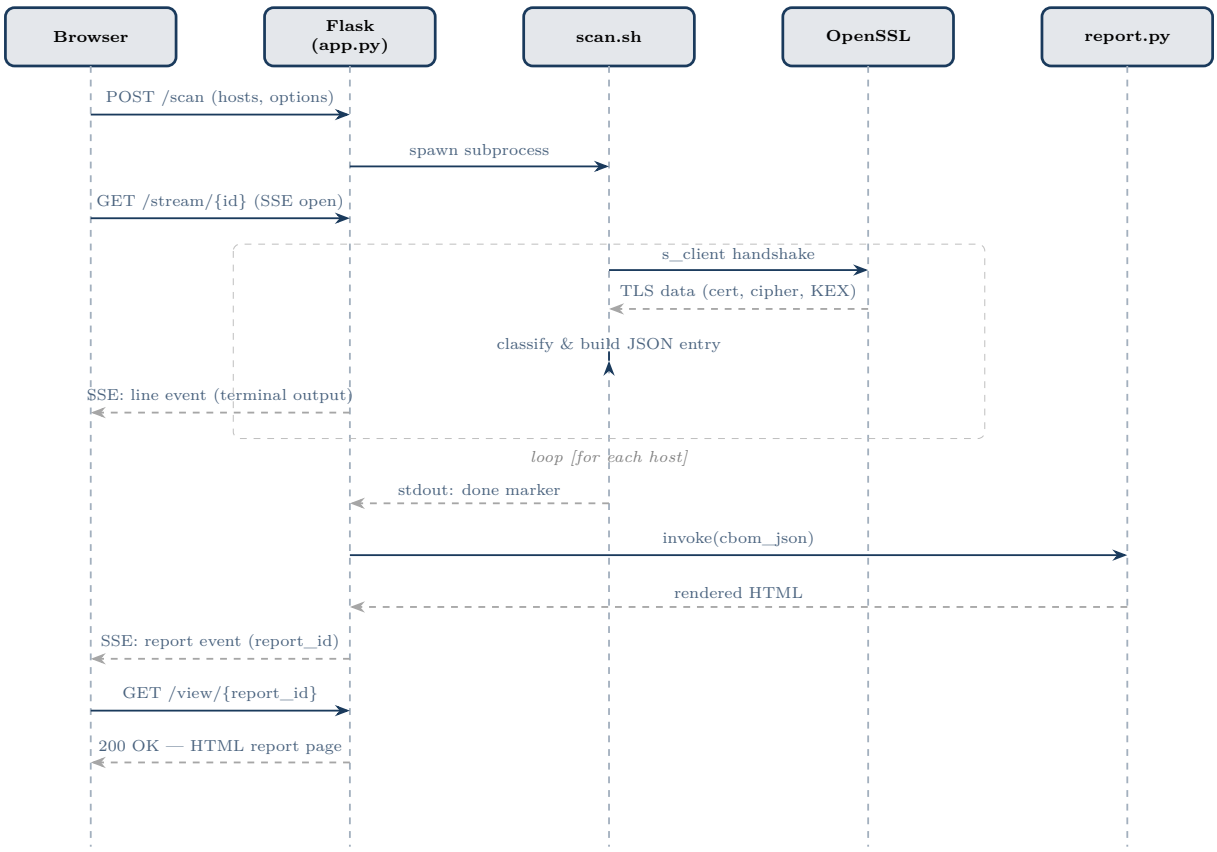
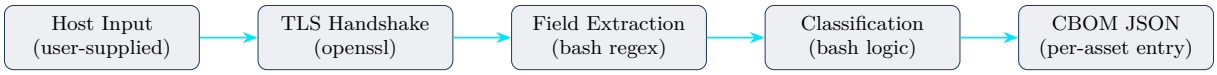


Figure 6.2: Sequence Diagram — End-to-end scan and report flow

6.3.2 CBOM Assembly



6.4 Deployment Architecture

6.4.1 Docker Container

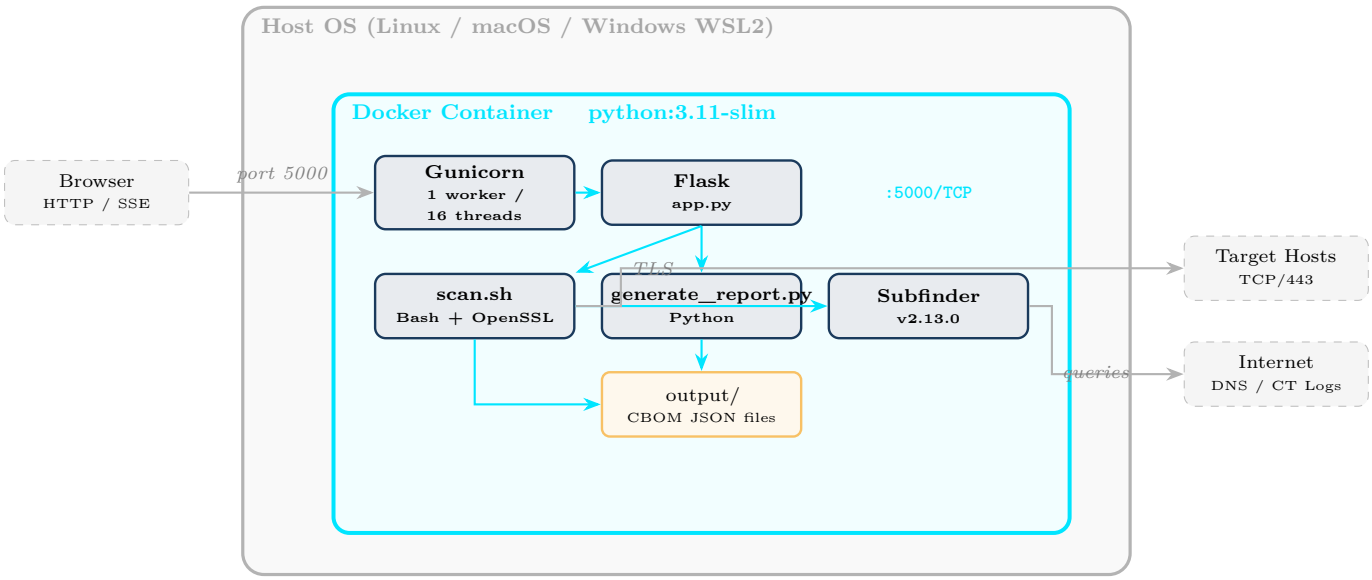


Figure 6.3: Deployment Diagram — Docker container with internal components and external connections

Table 6.1: Container Configuration

Parameter	Value
Base Image	`python:3.11-slim`
Exposed Port	`5000/TCP`
Entry Point	Gunicorn (`app:app`)
Workers	1 (single worker for shared memory state)
Threads per Worker	16 (gthread worker class)
Worker Timeout	660 seconds (11 minutes)
Key System Packages	`openssl`, `bash`, `coreutils`, `curl`
Bundled Binaries	Subfinder v2.13.0 (Linux x86_64)
Writable Volume	`output/` — CBOM JSON and report files

6.4.2 Concurrency Model

The application uses a **threaded concurrency model** within a single OS process:

- **Main thread (Gunicorn)** — handles HTTP requests, SSE connections, and scan lifecycle API calls.
- **Scan subprocess** — `scan.sh` runs as a child process; its PID is stored to enable stop-on-demand.
- **Background reaper thread** — a Python `daemon` thread that periodically evicts expired reports from in-memory storage.
- **SSE reader thread (per scan)** — a thread that reads scanner stdout and pushes lines into a `Queue` for SSE consumers.

Note: Because report state is held in a Python dict (in-memory), only a single Gunicorn worker is configured. Scaling to multiple workers would require an external cache (e.g. Redis) for shared state.

6.5 Key Algorithmic Decisions

6.5.1 Root Domain Approximation

The report generator groups hosts by root domain using a simple **eTLD+1 approximation**: the last two labels of the hostname (e.g. `example.com` from `api.example.com`). Port numbers are stripped before grouping.

6.5.2 Severity Ordering for Domain Groups

When a root domain has multiple subdomains with different quantum labels, the domain group's overall severity is determined by the **worst** label among all its hosts, using the ordering:

$$\text{FQS} = 0 \ < \ \text{PQC} = 1 \ < \ \text{NOT_PQC} = 3 \ < \ \text{CRITICAL} = 4$$

6.5.3 JSON Escaping in Bash

The scanner uses a custom `json_escape()` Bash function that performs character-by-character escaping of backslashes, double quotes, and control characters to ensure CBOM JSON is always valid — a critical requirement given that certificate subject fields can contain arbitrary Unicode and punctuation.

7 Other Requirements and Constraints

7.1 Legal and Ethical Constraints

- The PQC CBOM Scanner performs **active TLS interrogation** of target hosts. Users are responsible for ensuring they have appropriate authorisation to scan any host they supply. Unauthorised scanning of third-party servers may violate the Computer Misuse Act 1990 (UK), 18 U.S.C. § 1030 (US CFAA), or equivalent legislation in other jurisdictions.
- The tool does **not** exploit any vulnerability; it initiates standard TLS handshakes identical to those initiated by a web browser.
- The application does **not** store, transmit, or exfiltrate private keys or session secrets from target hosts.
- Certificate transparency log queries performed by Subfinder are a normal and publicly expected behaviour of subdomain enumeration; however, users should be aware that Subfinder's DNS queries may be visible to the DNS resolver.

7.2 Standards Compliance

Table 7.1: Standards and Compliance References

Standard	Relevance
NIST FIPS 203	ML-KEM standard; defines PQC key exchange algorithms recognised as *pqc_hybrid* or *pqc_pure*.
NIST FIPS 204	ML-DSA standard; defines PQC signature algorithms that classify a certificate as *pqc*.
NIST FIPS 205	SLH-DSA standard; additional PQC signature scheme recognised in certificate classification.
RFC 8446	TLS 1.3 specification; defines handshake fields parsed by the scanner.
IEEE 830-1998	This SRS document conforms to this recommended practice for software requirements specifications.
OWASP Top 10 A02	Cryptographic Failures category; the scanner identifies conditions matching this vulnerability class.

7.3 Resource Constraints

- **Memory**: In-memory report storage is bounded by TTL eviction (default 1 hour). For scan results of 500+ hosts, peak RSS memory usage should remain under 512 MB.
- **Disk**: The `output/` directory grows with each scan. Users are responsible for periodic clean-up; no automatic purging of CBOM JSON files is implemented.
- **Network bandwidth**: Each TLS handshake consumes < 5 KB of data. Subfinder subdomain discovery may issue hundreds of DNS queries per root domain.
- **CPU**: Scanning is I/O-bound (network timeouts dominate); CPU usage is expected to remain low even for large host lists.

7.4 Known Limitations

- **OpenSSL PQC Support**: Standard OpenSSL < 3.5 does not expose ML-KEM key exchange in `s_client` output. Hosts using X25519Kyber768 may report as classical key exchange on older OpenSSL builds.
- **Single-Worker Constraint**: The single Gunicorn worker means the application is not horizontally scalable without architectural changes (shared state store).
- **Protocol Scope**: Only TLS-based connections are assessed. SSH, IPsec, S/MIME, and application-layer cryptography are out of scope for v1.0.
- **Certificate Chain**: Only the leaf (end-entity) certificate is fully parsed. Intermediate CA certificates are counted for chain depth but not individually assessed.
- **Subfinder Architecture**: The bundled binary is Linux x86_64 only.
- **Report Persistence**: Reports are in-memory only and are lost on server restart. Users should download the HTML report before restarting.

7.5 Future Enhancements (Roadmap)

The following capabilities are considered for future releases:

Table 7.2: Potential Future Enhancements

#	Feature	Description
R1	SSH/IPsec Scanning	Extend CBOM coverage to non-TLS cryptographic channels.
R2	CI/CD GitHub Action	Publish a GitHub Action that runs the scanner on deployment and fails the pipeline if any host is *Critical*.
R3	Persistent Storage	SQLite or PostgreSQL backend for multi-session report history and trend analysis.
R4	PDF Report Export	Generate a PDF version of the HTML report using headless Chrome or WeasyPrint.
R5	SBOM/CBOM Integration	Export CBOM in CycloneDX JSON format for integration with existing SBOM toolchains.
R6	ARM64 Docker Image	Multi-arch Docker build with appropriate Subfinder binary for ARM64 hosts.
R7	Scheduled Scanning	Cron-triggered periodic scans with email or webhook notifications on status changes.
R8	OpenSSL 3.5 PQC	Leverage native ML-KEM support in OpenSSL 3.5 for more accurate key exchange detection.

8 Appendix

8.1 CBOM JSON Schema

The following describes the structure of the CBOM JSON file produced by the scanner. Fields marked `[opt]` are optional (present only when applicable).

```
{
  "scan_start":  "<ISO-8601 timestamp>",
  "scan_end":    "<ISO-8601 timestamp>",
  "host_count":  <integer>,
  "assets": [
    {
      "host":    "<hostname or IP>",
      "port":    <integer>,

      "connection": {
        "tls_version":    "<TLS 1.0 | 1.1 | 1.2 | 1.3 | unknown>",
        "cipher_suite":   "<cipher name>",
        "verified":       <true | false>,
        "error":          "<error message>"  [opt]
      },

      "key_exchange": {
        "type":      "<pqc_hybrid | pqc_pure | classical | unknown>",
        "algorithm": "<X25519 | X25519Kyber768 | ML-KEM-768 | ...>"
      },

      "certificate": {
        "subject":         "<Distinguished Name>",
        "issuer":          "<Distinguished Name>",
        "algorithm":       "<RSA | ECDSA | ML-DSA | SLH-DSA | ...>",
        "key_bits":        <integer>,
        "type":            "<pqc | rsa | ecdsa | unknown>",
        "not_before":      "<date string>",
        "not_after":       "<date string>",
        "days_until_expiry": <integer>,
        "chain_depth":     <integer>
      },

      "symmetric": {
        "cipher": "<AES-256-GCM | AES-128-GCM | ChaCha20 | 3DES | ...>",
        "class":  "<safe | marginal | broken | unknown>"
      },

      "quantum": {
        "label":           "<FULLY_QUANTUM_SAFE | PQC_READY |
                            NOT_PQC_READY | CRITICAL>",
        "display":         "<[FQS] Fully Quantum Safe | [PQC] PQC Ready |
                            [NOT] Not PQC Ready | [CRIT] Critical>",
        "vulnerabilities": ["<string>", ...],
        "recommendations": ["<string>", ...]
      },

```

```
      "scan_timestamp": "<ISO-8601 timestamp>"
    }
  ]
}
```

Listing 8.1: CBOM JSON Schema

8.2 Quantum Classification Decision Table

The following truth table defines the mapping from component classifications to quantum readiness labels. Priority is evaluated top-to-bottom; the first matching row determines the label.

Table 8.1: Classification Decision Table

Key Exchange	Certificate	Symmetric	Label	Display
Any	Any	broken	CRITICAL	✗ Critical
pqc_*	pqc	safe	FULLY_QUANTUM_SAFE	★ FQ Safe
pqc_*	rsa/ecdsa	safe	PQC_READY	▼ PQC Ready
pqc_*	pqc	marginal	PQC_READY	▼ PQC Ready
classical	Any	safe/marginal	NOT_PQC_READY	◇ Not Ready
Any	rsa/ecdsa	marginal	NOT_PQC_READY	◇ Not Ready
unknown	Any	Any	NOT_PQC_READY	◇ Not Ready

8.3 Environment Variables Reference

Table 8.2: Supported Environment Variables

Variable	Default	Description
PORT	5000	TCP port on which the Flask application listens.
REPORT_TTL	3600	Seconds before an in-memory report is evicted (1 hour).
PQC_CBOM_OUT	auto-generated	Full path for the CBOM JSON output file; set automatically by `app.py` when launching `scan.sh`.

8.4 Document Revision History

Table 8.3: Revision History

Version	Date	Author	Description
0.1	2026-03-01	quantumCheck Team	Initial draft — introduction and scope.
0.5	2026-03-10	quantumCheck Team	Added functional requirements, architecture, and interface sections.
1.0	2026-03-20	quantumCheck Team	Final draft for hackathon submission; all sections complete.

8.5 Glossary of Quantum Computing Threats

Shor's Algorithm A quantum algorithm (1994) that can factor large integers and compute discrete logarithms in polynomial time, breaking RSA, ECDSA, ECDH, and Diffie-Hellman. Practical execution requires a cryptographically-relevant quantum computer (CRQC) with millions of logical qubits.

Grover's Algorithm A quantum search algorithm that provides a quadratic speedup for unstructured search, effectively halving the bit-security of symmetric ciphers. AES-128 is reduced to 64-bit security; AES-256 retains 128-bit security.

CRQC Cryptographically Relevant Quantum Computer — a hypothetical future device capable of running Shor's algorithm at scale to break current public-key cryptography.

Harvest Now, Decrypt Later (HNDL) A threat strategy where adversaries capture and store encrypted ciphertext today with the intention of decrypting it once a CRQC becomes available. TLS key exchange is the primary target.

Hybrid Key Exchange A TLS key exchange that combines a classical algorithm (e.g. X25519) with a PQC algorithm (e.g. Kyber) so that security holds as long as *either* algorithm remains unbroken. Represented by `X25519Kyber768` in TLS extensions.